Predicting Real-Estate Prices and Price Tiers from Scratch:
Decision Tree and Pegasos SVM Tested on bina.az

Alasgarov Teymur Heydarov Tofiq Salahov Parvin

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**Estimating property prices is hard in a market like Azerbaijan's. Prices are very skewed, location matters a lot, and listings scraped from websites are messy. In this project we built two models completely from scratch using only NumPy. The first is a decision tree (it uses MSE for regression, and Gini impurity or entropy for classification). The second is a Support Vector Machine trained with the Pegasos algorithm, with Random Fourier Features (RFF) added so it can learn non-linear patterns.

We tested both models on 93,150 cleaned listings from `bina.az`, with two tasks: predicting the log-price (Task A), and classifying whether a property is above or below the median price (Task B).

On Task A, our tree got an RMSE of 0.5036 ($R^2 = 0.5992$). scikit-learn got 0.5026 ($R^2 = 0.6007$), so the difference is only 0.0010. On Task B, our tree got 90.79% test accuracy ($F_1 = 0.9131$, ROC-AUC 0.9675), which matches scikit-learn ($F_1 = 0.9131$, ROC-AUC 0.9674). Our Pegasos SVM with RFF reached 84.39% accuracy ($F_1 = 0.8512$, ROC-AUC 0.9271), and its ROC-AUC was higher than the standard linear SGD baseline (0.9198). 5-fold cross-validation shows that the results are stable ($F_1 = 0.9123 \pm 0.0016$).

Our code, tuning experiments, and scripts are available at `github.com/ParvinSalahov/ML_Final_Project` (tag `v1.0-final`).

***Index Terms*—**Decision tree, Support Vector Machine, Pegasos, Random Fourier Features, real-estate valuation, reproducible benchmarking.

I. Introduction

Automated valuation models (AVMs) are used for mortgage decisions, market transparency, and risk management. But raw listings from `bina.az` are tricky to work with. Prices go from 11 AZN up to 627,130 AZN, the descriptions are written in free-form Azerbaijani with inconsistent wording, and many categories are very imbalanced.

In this project we wanted to show that models we code ourselves from the basic math can give results close to professional libraries. Our main contributions are:

- **From-scratch algorithms**: A recursive binary decision tree (MSE, Gini, and entropy) and a hinge-loss SVM trained with Pegasos, using Random Fourier Features to get non-linear decision boundaries. Both are vectorised with NumPy.
- **Leakage-free features**: Careful data cleaning, extracting information from the text, frequency encoding, and computing every statistic only on the training data.
- **Benchmarking**: We compare against scikit-learn on 93,150 samples, with 5-fold cross-validation, residual analysis, and hyperparameter sweeps.
- **Reproducibility**: One command (`python -m src.run_all`) recreates all 19 figures and tables.

II. Related Work / Background

Decision trees split the feature space with simple yes/no rules, choosing the split that makes the groups as pure as possible [2]. They are greedy, but they handle mixed numeric and categorical features, non-linear patterns, and different feature scales without needing standardization.

A soft-margin SVM looks for the hyperplane with the widest margin while allowing a few violations (the hinge loss). Instead of solving the usual dual problem with SMO, Shalev-Shwartz et al. [1] introduced Pegasos, which works directly on the primal objective with stochastic sub-gradient steps. It reaches an ε-accurate solution in $\mathcal{O}(1/(\lambda\varepsilon))$ iterations, no matter how large the dataset is. To capture non-linear patterns in the market, we map features with Random Fourier Features [3], which approximate the Gaussian RBF kernel $\mathcal{K}(\mathbf{x},\mathbf{z}) = \exp(-\gamma\|\mathbf{x}-\mathbf{z}\|^2)$.

III. Data and Preprocessing

A. Dataset Cleaning

The raw dataset has 100,775 listings scraped from `bina.az`. We cleaned it in these steps:

1. **Removing duplicates**: Removing exact duplicate listings left 100,773 rows.
2. **Extracting information**: Regular expressions pull the number of rooms and the renovation status (*temir*) out of the free-text description and attributes fields.
3. **Filtering extreme rows**: We removed entries that make no physical sense (for example zero area, non-residential anomalies, and corrupted outliers). This left 93,150 valid records with 28 features.
4. **Transforming the target**: Raw prices are strongly skewed (skewness 0.877). We use $y = \log(1 + \text{price})$, which shrinks the huge price range and stabilizes the variance (skewness becomes -1.340).
5. **Preventing leakage**: We dropped administrative IDs, duplicate URLs, and price-per-unit ratios, because they

leak the target. Imputation values, frequency encodings, and scaling statistics were all computed on the training split only.

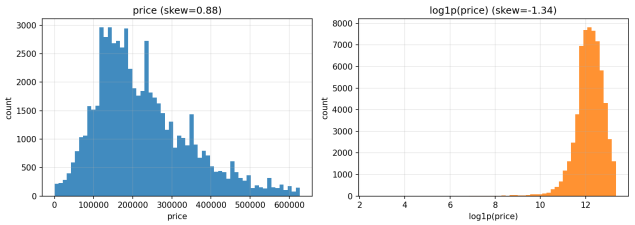


Figure 1. Distribution of raw prices (left, strongly skewed) compared with log-transformed prices $\log(1+\text{price})$ (right), on the training data.

Table 1. Dataset summary and split statistics.

Property	Absolute Count	Share
Raw input rows	100,775	100.0%
After deduplication	100,773	100.0%
Total clean analytical rows	93,150	100.0%
Train split	62,922	67.5%
Validation split	15,114	16.2%
Test split	15,114	16.2%
Engineered input features	28	—
Task B train median price	200,000 AZN	$\log p = 12.21$
Task B balance (Tr/Val/Te)	49.4 / 54.1 / 54.2	(% Premium)

B. The Two Tasks

We formulate two complementary learning challenges:

- **Task A (regression)**: Predict the log-price $y = \log(1+\text{price})$.
- **Task B (classification)**: Decide whether a property is premium (1) or standard (0), using the training median as the cutoff ($\tau = 12.2061$, about 200,000 AZN). The classes are fairly balanced; the test set has 54.2% premium and 45.8% standard.

IV. Methodology

A. Decision Tree (From Scratch)

The tree builds its splits greedily and recursively. For every feature j and every candidate threshold θ, we measure how much the split reduces impurity:

$$\Delta I(S, j, \theta) = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R). \tag{1}$$

Here $S_L = \{(\mathbf{x}, y) \in S \mid x_j \le \theta\}$ is the group of samples with $x_j \le \theta$, and $S_R = S \setminus S_L$ is everything else. In other words, we compare the impurity before the split with the weighted impurity of the two groups after it.

For regression (Task A), the impurity is the mean squared error around the node's mean:

$$I_{\text{MSE}}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2, \quad \bar{y}_S = \frac{1}{|S|} \sum_{i \in S} y_i. \tag{2}$$

For classification (Task B), we implemented two options, where $p_k = \frac{1}{|S|} \sum_{i \in S} \mathbb{I}(y_i = k)$ is the share of samples of class k in the node:

$$I_{\text{Gini}}(S) = 1 - \sum_{k=0}^{1} p_k^2, \tag{3}$$

$$I_{\text{Entropy}}(S) = -\sum_{k=0}^{1} p_k \log_2(p_k + \varepsilon). \tag{4}$$

A node stops splitting when any of these happens: the depth reaches $d_{\max}$, the node has fewer than $2 \cdot n_{\text{leaf}}$ samples, or a split would leave fewer than n_{leaf} samples on either side. A leaf predicts the mean $\bar{y}$ for regression, and the class probabilities $[p_0, p_1]$ for classification.

B. Support Vector Machine with Pegasos (From Scratch)

For binary classification with labels $y_i \in \{-1, +1\}$, we minimize the regularized hinge-loss objective:

$$J(\mathbf{w}, b) = \frac{\lambda}{2} \|\mathbf{w}\|_2^2 + \frac{1}{n} \sum_{i=1}^{n} \max\left(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right). \tag{5}$$

The value $\lambda > 0$ controls the regularization ($C \approx 1/(n\lambda)$). At step t the learning rate is $\eta_t = \frac{1}{\lambda t}$, and we pick one random sample $(\mathbf{x}_t, y_t)$. If it violates the margin, meaning $y_t(\mathbf{w}_t^\top \mathbf{x}_t + b_t) < 1$, we update:

$$\mathbf{w}_{t+1} = (1 - \eta_t \lambda)\mathbf{w}_t + \eta_t y_t \mathbf{x}_t, \tag{6}$$

$$b_{t+1} = b_t + \eta_t y_t. \tag{7}$$

Otherwise we only shrink the weights: $\mathbf{w}_{t+1} = (1 - \eta_t \lambda)\mathbf{w}_t$.

To get non-linear boundaries cheaply, we map the standardized features $\mathbf{x} \in \mathbb{R}^d$ into a D-dimensional random Fourier space $\mathbf{z}(\mathbf{x}) \in \mathbb{R}^D$:

$$z_j(\mathbf{x}) = \sqrt{\frac{2}{D}} \cos(\omega_j^\top \mathbf{x} + \beta_j). \tag{8}$$

The frequencies ω_j are drawn from a normal distribution $\mathcal{N}(\mathbf{0}, 2\gamma \mathbf{I})$, and the phase shifts β_j are drawn uniformly from $[0, 2\pi]$. This way we avoid building the huge $n \times n$ kernel matrix of the dual SVM.

V. Experimental Setup

A. Evaluation Metrics

All final scores come from the test split that the models never saw ($n = 15,114$):

- **Task A**: RMSE, MAE, and R^2.
- **Task B**: Accuracy, Precision, Recall, Macro F_1, ROC-AUC, and PR-AUC.

B. Hyperparameter Tuning

We tuned hyperparameters only on the validation set ($n = 15,114$):

- **Decision tree**: We tried max depth from 1 to 18, both criteria (Gini and entropy), and leaf size from 1 to 100. The best setting was criterion = entropy, $d = 12$, $n_{\text{leaf}} = 10$.
- **Pegasos SVM**: We tried λ from 10^{-5} to 1.0 and the RFF scale γ from 0.1 to 10.0, with 256 components and 30 epochs. The best setting was $\lambda = 10^{-4}$ ($C \approx 0.1589$) and $\gamma = 0.1$.

VI. Results and Benchmarking

A. Do Our Models Match scikit-learn?

Table 2 shows that our from-scratch decision tree gets very close to scikit-learn's optimized implementation. On Task A the RMSE differs by only 0.0010 (0.5036 vs 0.5026), and the R^2 values are almost the same (0.5992 vs 0.6007). On Task B the results are practically the same: the same F_1 (0.9131), the same PR-AUC (0.9671), and almost the same ROC-AUC (0.9675 vs 0.9674).

On Task B our Pegasos SVM with RFF reaches an F_1 of 0.8512. It ranks examples better than scikit-learn's linear SGD classifier (ROC-AUC 0.9271 vs 0.9198, PR-AUC 0.9375 vs 0.9255), which suggests that the non-linear Fourier features help. The SGD baseline is still slightly better on accuracy and F_1, though.

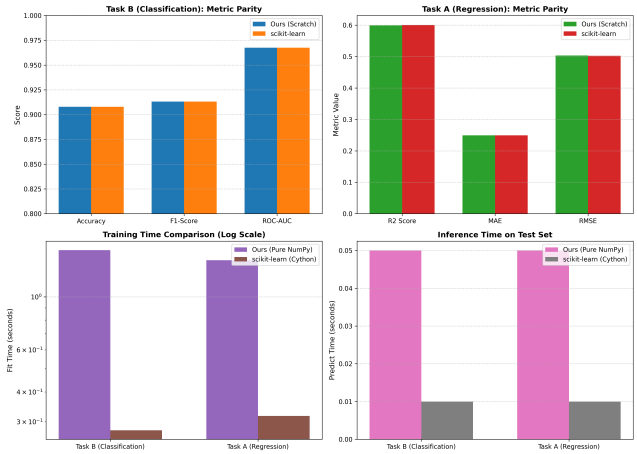


Figure 2. Comparison between our decision tree and scikit-learn on the Task A and Task B test splits.

B. Cross-Validation

To check that our results do not depend on one lucky split, we ran 5-fold cross-validation on the training data (62,922 samples):

- **Regression RMSE**: Ours 0.2719 ± 0.0083, scikit-learn 0.2715 ± 0.0086.
- **Regression R^2**: Ours 0.8059 ± 0.0085, scikit-learn 0.8064 ± 0.0088.
- **Classification F_1**: Ours 0.9123 ± 0.0016, scikit-learn 0.9124 ± 0.0013.
- **Classification ROC-AUC**: Ours 0.9714 ± 0.0012, scikit-learn 0.9713 ± 0.0010.

The standard deviations are small (below 0.002 for classification), so performance does not change much between folds.

C. Hyperparameter Sensitivity and Bias–Variance Tradeoff

Figure 3 shows how tree depth affects the model. For very deep trees ($d \geq 15$), training ROC-AUC goes up to 0.9996 while validation ROC-AUC falls to 0.9252, which is a clear sign of overfitting. Setting a minimum leaf size of $n_{\text{leaf}} = 10$ at $d = 12$ removes the overconfident leaves that contain a single sample, and raises validation ROC-AUC to 0.9660.

For the Pegasos SVM, the (λ, γ) grid shows that a very large regularization ($\lambda \geq 0.1$) over-smooths the model: the weights shrink to almost zero ($\|\mathbf{w}\| \to 0$) and F_1 drops to about 0. The best balance was a small λ with a moderate kernel scale ($\lambda = 10^{-4}$, $\gamma = 0.1$). It has a margin width of $2/\|\mathbf{w}\| = 0.1041$ and 23,840 margin support vectors, and gives the best validation F_1 of 0.8370.

D. Error Analysis by Property Type

When we split the Task A test predictions by property type, the model works very differently across categories:

- **Apartments**: Predictions are accurate and consistent for *Yeni tikili* ($n = 8,616$, RMSE 0.218, $R^2 = 0.838$) and *Köhne tikili* ($n = 2,753$, RMSE 0.201, $R^2 = 0.735$).
- **Commercial and land**: Errors are much larger for *Obyekt* ($n = 623$, RMSE 1.256, $R^2 = 0.062$) and *Torpaq* ($n = 737$, RMSE 1.483, $R^2 = 0.281$). Likely reasons are missing information such as street frontage, zoning permits, and access to utilities.

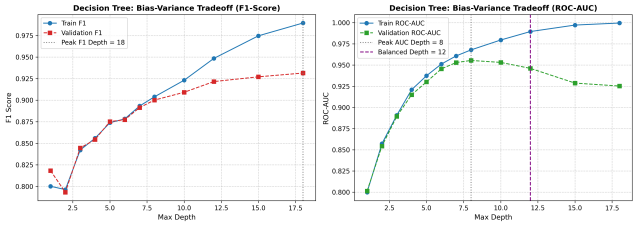


Figure 3. Bias-variance tradeoff for the depth sweep ($d \in [1, 18]$). Training ROC-AUC gets close to 0.9996, while validation ROC-AUC drops for deep trees because the tree memorizes the training leaves.

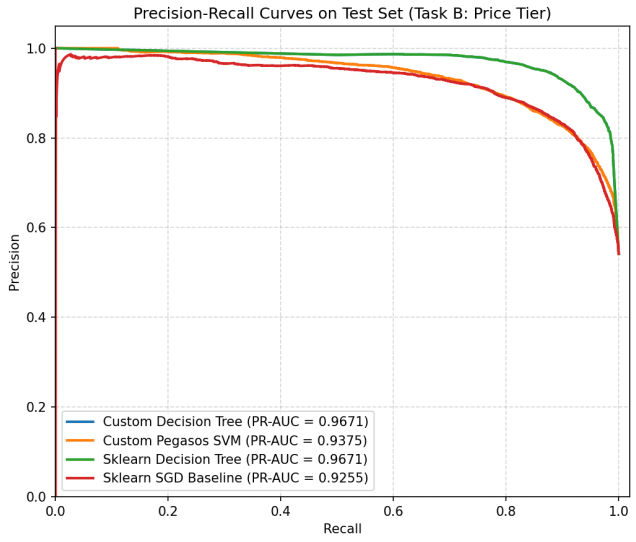


Figure 4. Precision-Recall curves on the test set for Task B. Both tree models keep high precision at high recall.

Table 2. Consolidated Headline Results on Unseen Test Split ($n = 15,114$).

	Task A: Log-Price Regression			Task B: Price-Tier Classification				Compute
Model	RMSE ↓	MAE ↓	R^2 ↑	Accuracy ↑	F_1-Score ↑	ROC-AUC ↑	PR-AUC ↑	Fit Time (s)
Ours: Decision Tree	**0.5036**	**0.2495**	**0.5992**	**0.9079**	**0.9131**	**0.9675**	**0.9671**	4.53 s
scikit-learn Decision Tree	0.5026	0.2493	0.6007	0.9078	0.9131	0.9674	0.9671	0.98 s
Ours: Pegasos SVM (RFF)	—	—	—	0.8439	0.8512	**0.9271**	**0.9375**	10.90 s
scikit-learn SGD Baseline	—	—	—	**0.8478**	**0.8578**	0.9198	0.9255	0.63 s
Bonus: Ours Ridge (Normal Eq.)	0.5172	0.3169	0.5773	—	—	—	—	0.09 s
Bonus: Ours SVR (Sub-gradient)	0.5086	0.3132	0.5911	—	—	—	—	8.03 s
Bonus: Ours Random Forest (46 trees)	0.4828	—	—	—	—	—	—	—
Bonus: Ours Gradient Boosting (200 trees)	**0.4552**	—	—	—	—	—	—	—

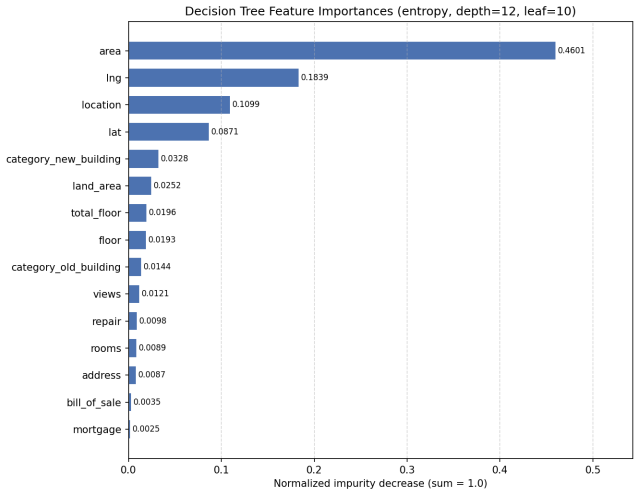


Figure 5. Top 15 features by Mean Decrease in Impurity (MDI). Living area, number of rooms, and the coordinates of premium Baku districts are the most important.

VII. Discussion and Limitations

Our implementations perform close to the scikit-learn baselines, but they have some limits:

1. **Axis-aligned splits**: A tree can only split along one feature at a time, so it has trouble with diagonal patterns such as price gradients across Baku.
2. **Limited random features**: Random Fourier Features allow non-linear boundaries, but $D = 256$ is limited by computing time, so very detailed local price patterns are only approximated.
3. **Rare listings**: For luxury properties (above 500,000 AZN) and sparse rural regions (for example Astara and Gedebey), there are few examples, so prediction errors are larger.

VIII. Conclusion

We built a recursive decision tree and a primal Pegasos SVM with Random Fourier Features from scratch, and tested them on real-estate data from `bina.az`. Our tree reaches results very close to scikit-learn ($F_1 = 0.9131$, and tree RMSE 0.5036 vs 0.5026). Cross-validation and the bias-variance experiments show that the results are stable. In the future we want to add spatial embeddings from geographic graph neural networks, and oblique (diagonal) splits in the tree.

Tools and Acknowledgements

We used AI coding assistants for template formatting, exploratory visualization scripts, and improving the documentation. The data comes from public listings on `bina.az`.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[2] L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen, *Classification and Regression Trees*, CRC press, 1984.

[3] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 20, 2007.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.